5

Unit 5: Input-Output and Memory Organization

Computer Organization and Architecture

CHANDRAS EDU

Never stop learning

Agenda

5.1 Input-Output Interface
5.2 Modes of Transfer
5.3 Priority Interrupt
5.4 Direct Memory Access
5.5 Memory Hierarchy
5.6 Main Memory
5.7 Auxiliary Memory
5.8 Associative Memory
5.9 Cache Memory

5.1 Input-Output Interface

IN SIMPLE TERMS

An interface is a translator between the fast, electronic CPU and a slower, different peripheral. It holds the data, commands and status in its own registers so the two sides can work at their own speeds.

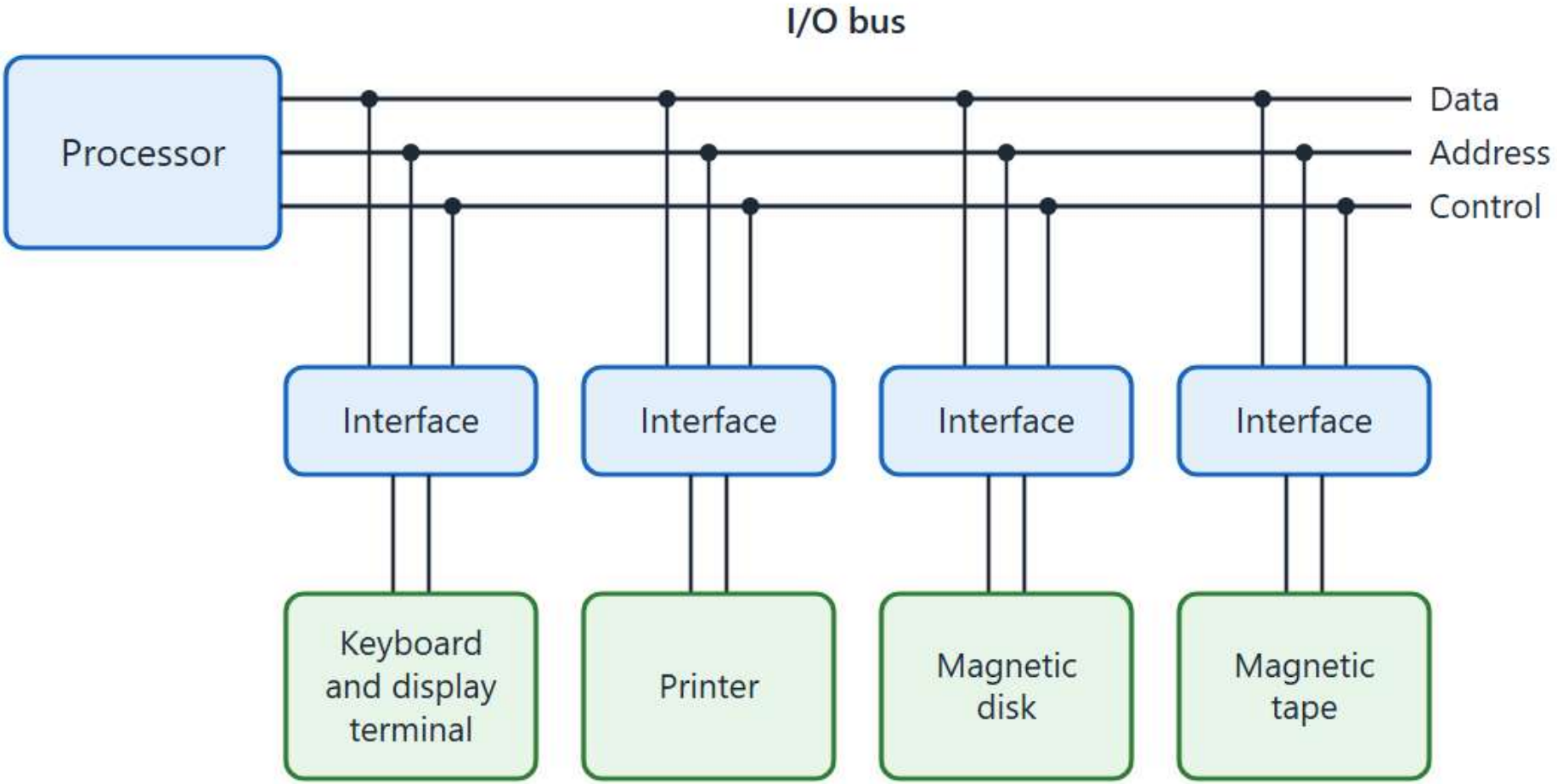
Why an interface is needed

- Peripherals differ from the CPU in four ways
- **Signal type:** electromechanical or electromagnetic devices versus a purely electronic CPU
- **Speed:** peripherals are far slower, so synchronisation is needed
- **Data codes:** different word lengths or encoding, e.g. 7-bit ASCII in an 8-bit byte
- **Operating modes:** each peripheral has its own command set
- Each peripheral gets an **interface unit** between the CPU bus and the device controller

The I/O bus and the four commands

- A shared **I/O bus** carries data lines, address lines and control lines
- Each interface monitors the address lines and activates when its own address appears
- **Control command:** activates the peripheral and sets its mode (e.g. rewind tape)
- **Status command:** reads the status register: ready or error?
- **Data output command:** CPU data bus → interface buffer register
- **Data input command:** interface buffer register → CPU data bus

I/O bus and devices

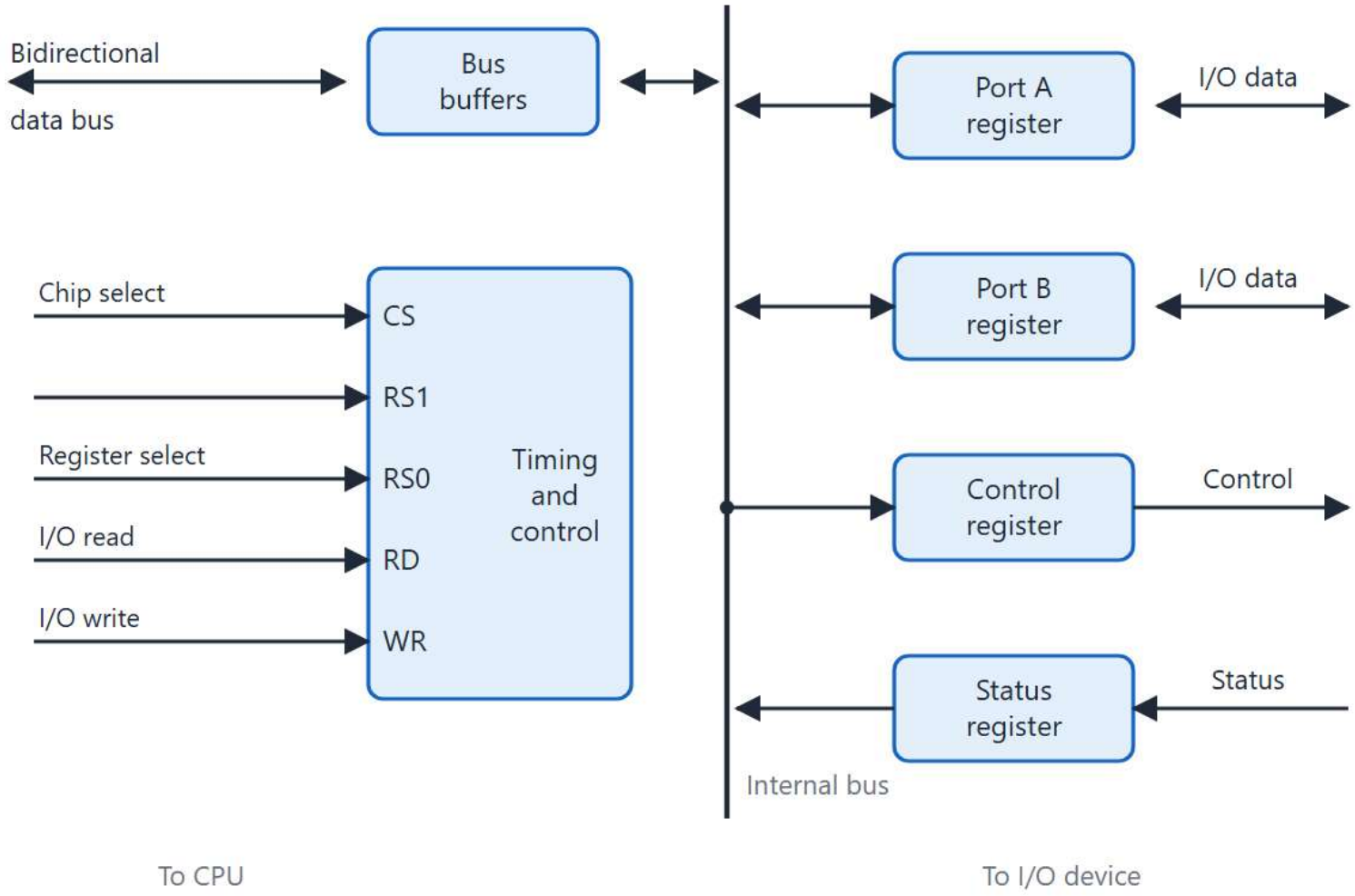


Connection of I/O bus to input-output devices. Adapted from M. Morris Mano, Computer System Architecture, Fig. 11-1.

Isolated versus memory-mapped I/O

- **Isolated I/O:** separate address spaces for memory and I/O
- Dedicated I/O read and I/O write lines; the CPU uses IN/OUT instructions; addresses cannot clash
- **Memory-mapped I/O:** interface registers occupy part of the memory address space
- The CPU uses ordinary load and store instructions: more flexible, but less memory address range
- **Advantage:** all memory-reference instructions (AND, OR, shift, compare…) can work directly on I/O data

Interface unit registers



I/O interface unit registers and bus connections. Adapted from M. Morris Mano, Computer System Architecture, Fig. 11-2.

Register selection

CS	RS_1	RS_0	Register selected
0	×	×	None: data bus in high impedance
1	0	0	Port A register
1	0	1	Port B register
1	1	0	Control register
1	1	1	Status register

Register selection for the interface in the figure on the previous slide

5.2 Modes of Transfer

IN SIMPLE TERMS

There are three ways to move I/O data: the CPU keeps checking (programmed I/O), the device taps the CPU on the shoulder when ready (interrupt), or a DMA controller moves whole blocks while the CPU does other work.

Three modes of I/O data transfer

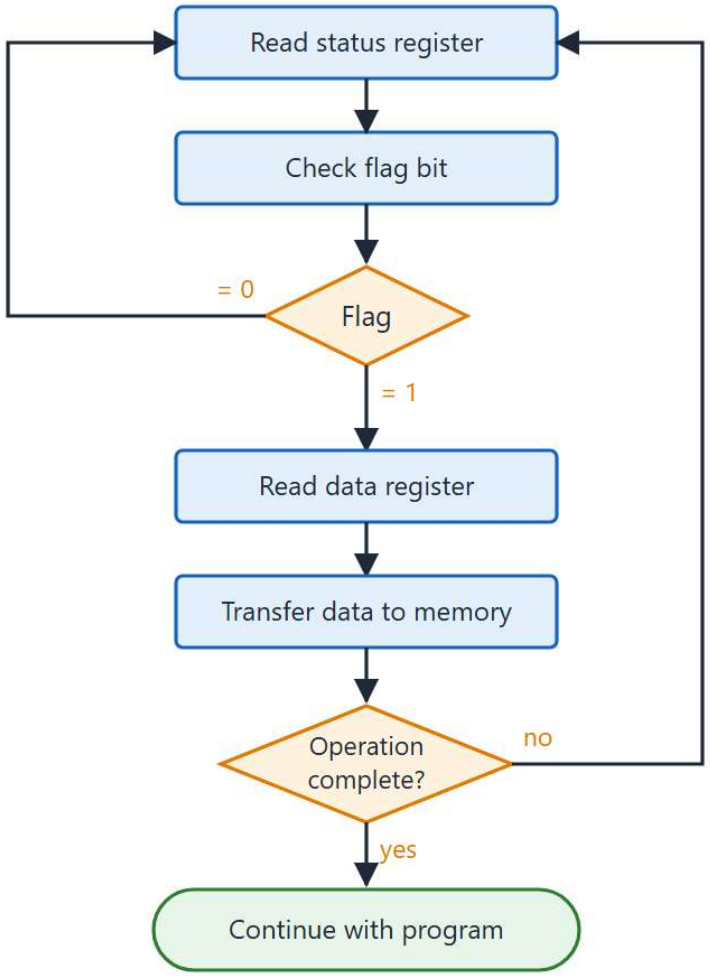
Programmed I/O	Interrupt-initiated I/O	Direct memory access
CPU checks the flag again and again	Device raises an interrupt when ready	DMA controller moves whole blocks
Simple, no extra hardware	CPU saves state, runs service routine, returns	CPU gives up the bus, continues other work
CPU can do no other work	CPU is free between transfers	Fastest; needs DMA hardware

The three modes of I/O data transfer — overview

Mode 1: programmed I/O (polling)

- The interface sets a **flag bit F** when a new byte is ready; the CPU does everything else
- **Step 1:** read the status register
- **Step 2:** if F = 0, go back to step 1 (busy-wait)
- **Step 3:** when F = 1, read the data register, store the byte in memory, clear the flag
- **Efficiency problem:** 1 byte per 10,000 µs and 1 poll per µs means 10,000 checks per byte, 9,999 wasted
- Practical only for very fast devices or embedded systems with nothing else to do

Programmed I/O flowchart



- Read the status register
- If F = 0, go back and read it again (busy-wait)
- When F = 1, read the data register
- Store the byte in memory and clear the flag

Flowchart for CPU program to input data (programmed I/O). Adapted from M. Morris Mano, Computer System Architecture, Fig. 11-11.

Mode 2: interrupt-initiated I/O

- The interface may **interrupt** the CPU when data is ready; the CPU runs its main program meanwhile
- The interface asserts the interrupt line INT
- At the end of the current instruction the CPU saves its PC on the stack and branches to the **interrupt service routine (ISR)**
- The ISR reads the data register, stores the byte, clears the flag; the CPU then resumes
- **Non-vectored:** fixed branch address; the CPU then polls devices to find the interrupter
- **Vectored:** the device puts a vector (its ISR address or a pointer) on the data bus; no polling

Mode 3: direct memory access (DMA)

- For high-speed devices (disk, network) one interrupt per byte is too much overhead
- A **DMA controller** handles the whole block
- **1.** CPU programs the controller: start address, byte count, direction
- **2.** CPU releases the bus and continues (if it has a cache or separate bus), or halts bus access briefly
- **3.** Controller transfers the block: address +1, count −1 per byte
- **4.** At count zero the controller interrupts the CPU

Comparison of the three I/O transfer modes

Property	Programmed I/O	Interrupt-initiated	DMA
CPU during transfer	Busy-waiting (polls)	Runs other code; ISR on interrupt	Continues; DMA owns bus
Hardware needed	Interface only	Interface + interrupt logic	Interface + DMA controller
Transfer unit	1 byte per poll	1 byte per interrupt	Block (many bytes)
Suitable for	Slow/simple devices	Medium-speed devices	High-speed block devices
CPU efficiency	Lowest	Good	Highest

5.3 Priority Interrupt

IN SIMPLE TERMS

When several devices interrupt at once, the computer serves the most urgent first. The order is set either by a program checking devices in turn, by their position in a chain (daisy chain), or by a priority encoder.

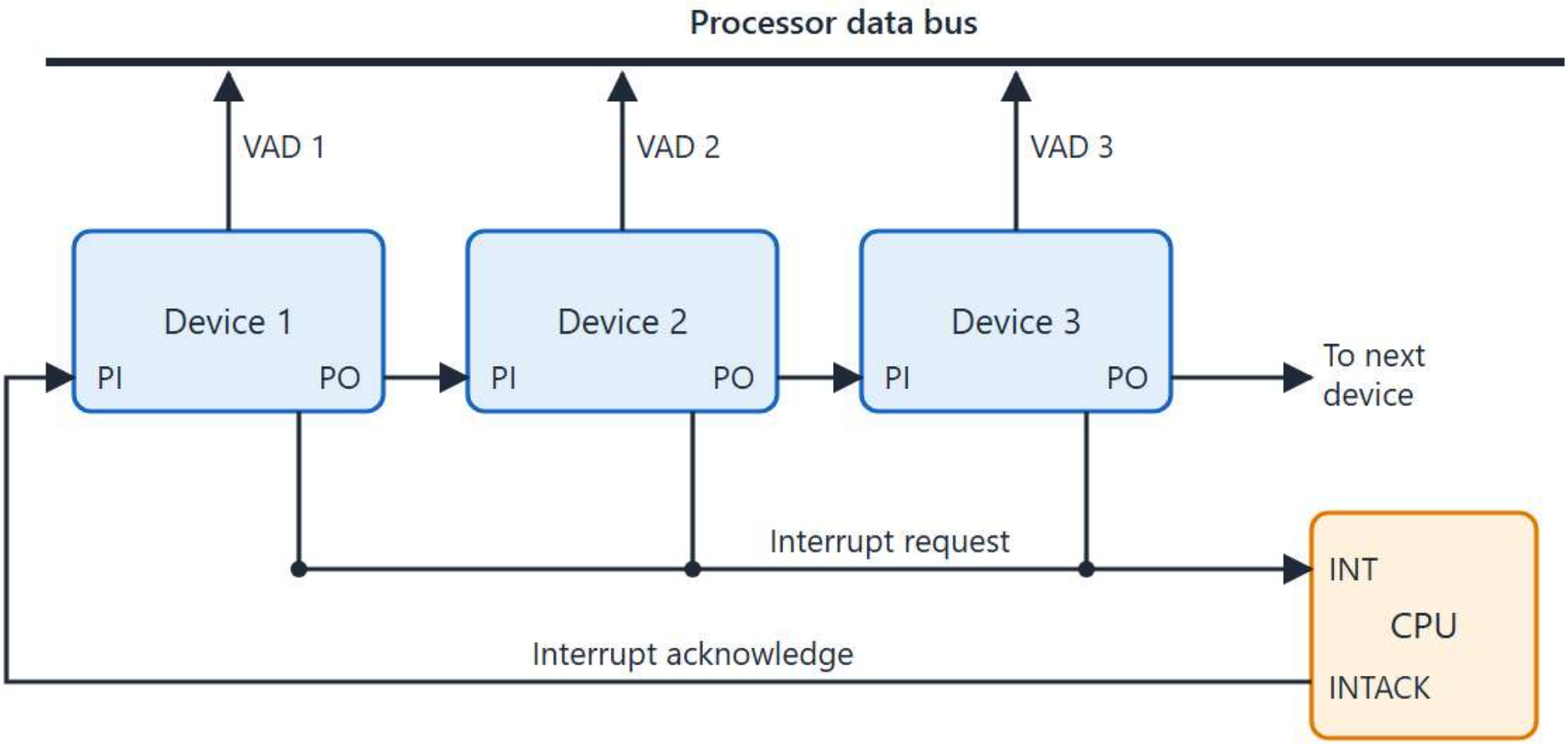
The priority problem and software polling

- Each device gets a priority; if several interrupt together, the highest is served first
- Disks get high priority (a missed transfer can lose data); keyboards get low priority
- Two approaches: **software polling** or **hardware**
- Software polling: all devices share one interrupt line and one common service routine
- The routine tests each device's status flag, highest priority first; the first flag set is served
- Simple and flexible, but slow when there are many devices

Hardware priority: daisy chain

- Devices form a serial chain; each has **Priority In (PI)** and **Priority Out (PO)**
- The CPU sends **INTACK** into Device 1 at its PI
- A device with no pending interrupt passes it on: its PO feeds the next PI
- The first device with a pending interrupt intercepts INTACK: PO = 0 and it puts its **vector address (VAD)** on the data bus
- The CPU reads VAD and jumps to that service routine
- Priority is by position: closest to the CPU is highest

Daisy-chain priority interrupt



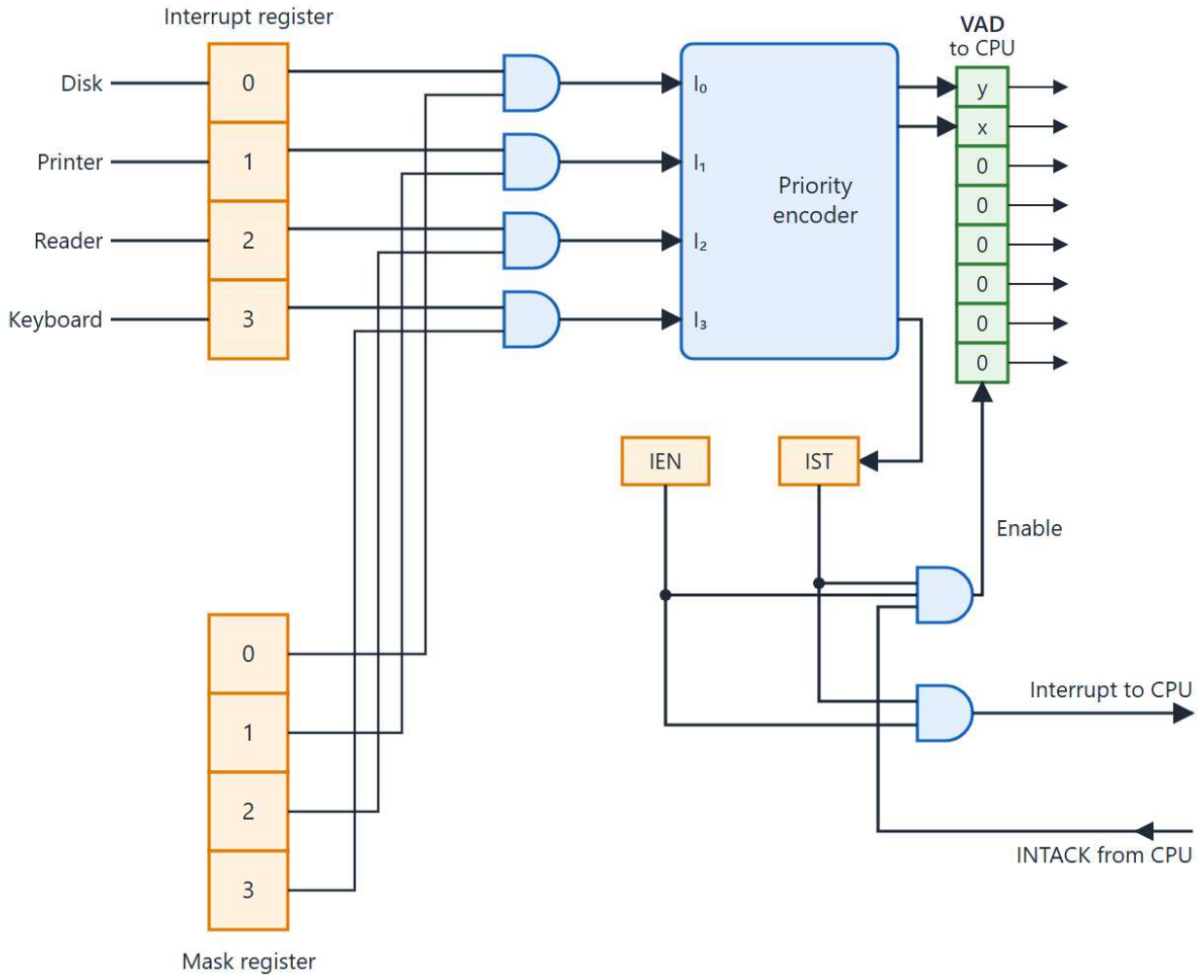
Daisy-chain priority interrupt (three devices). Adapted from M. Morris Mano, Computer System Architecture, Fig. 11-12.

One daisy-chain stage

RF is the request flip-flop, set by the device's interrupt request.

PI	RF	PO	VAD
0	any	0	Disabled: INTACK blocked
1	0	1	Disabled: INTACK passed on
1	1	0	Enabled: this device is served

Parallel priority interrupt hardware



- Four devices: disk, printer, reader, keyboard (priority 0 to 3)
- Each interrupt bit is ANDed with its mask bit
- The priority encoder outputs two bits, x and y, which form part of the vector address (VAD) of the highest-priority device
- I_0 (disk) has the highest priority

Priority interrupt hardware (interrupt register, mask register, priority encoder). Adapted from M. Morris Mano, Computer System Architecture, Fig. 11-14.

Parallel method: components

- **Interrupt register:** one bit per device, set by its request, cleared by program
- **Mask register:** one bit per device, set or cleared by program; masked (0) devices are ignored
- **Priority encoder:** takes I_0–I_3 and gives a 2-bit VAD for the highest-priority active device
- **IST** (interrupt status flip-flop): set when any unmasked interrupt is active
- **IEN** (interrupt enable): global switch; IST AND IEN = interrupt to the CPU
- **INTACK** from the CPU enables the VAD buffers so the winner’s address reaches the data bus

5.4 Direct Memory Access

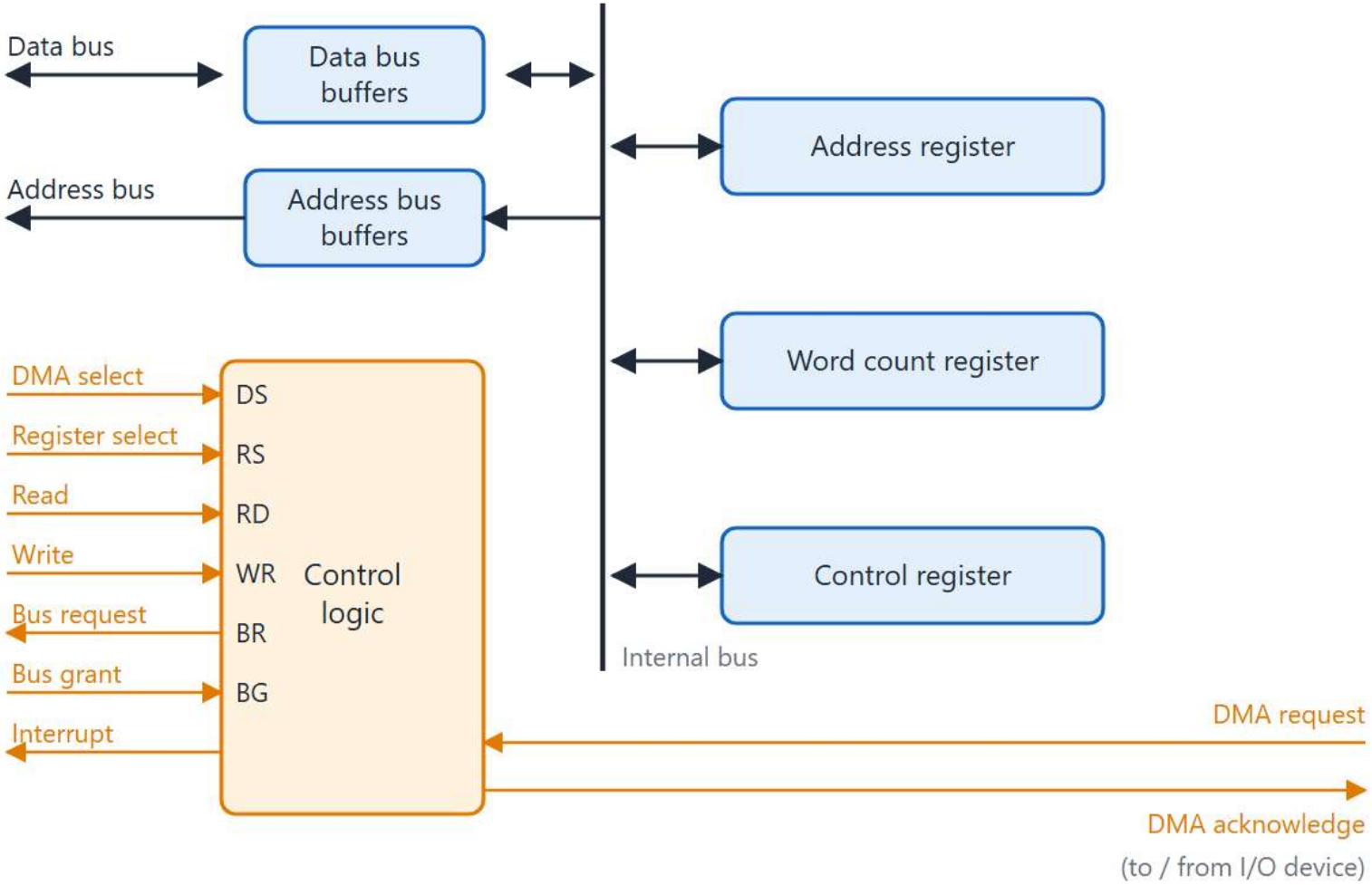
IN SIMPLE TERMS

Instead of the CPU carrying every byte between a fast device and memory, a DMA controller borrows the memory bus and moves the data itself. The CPU only sets it up at the start and gets one interrupt at the end.

The DMA idea and its registers

- In programmed and interrupt-initiated I/O every word flows through the CPU, a bottleneck for fast devices
- The **DMA controller** takes temporary ownership of the memory bus; data flows directly between device and memory
- **Address Register (AR):** starting memory address; +1 after each word
- **Word Count Register (WC):** words to transfer; −1 after each word; at zero the CPU is interrupted
- **Control/Status Register:** direction and start command in; done and error status out

DMA controller

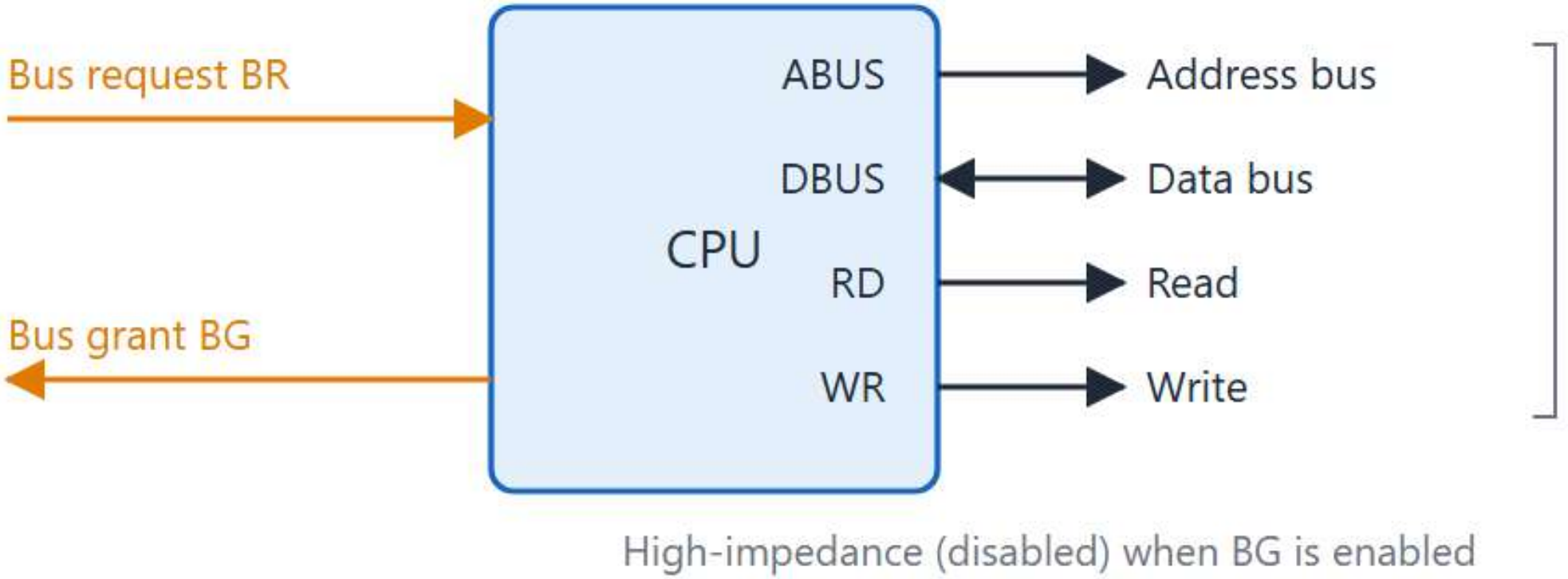


Block diagram of DMA controller. Adapted from M. Morris Mano, Computer System Architecture, Fig. 11-18.

Bus request and bus grant

- The DMA controller and the CPU share the memory bus
- **Bus Request (BR):** the controller asks for the bus
- **Bus Grant (BG):** the CPU answers after finishing its current bus cycle
- While BG is active the CPU puts its address, data and read/write lines in high impedance
- The controller transfers, then releases the bus; the CPU resumes normal operation

CPU bus signals for DMA

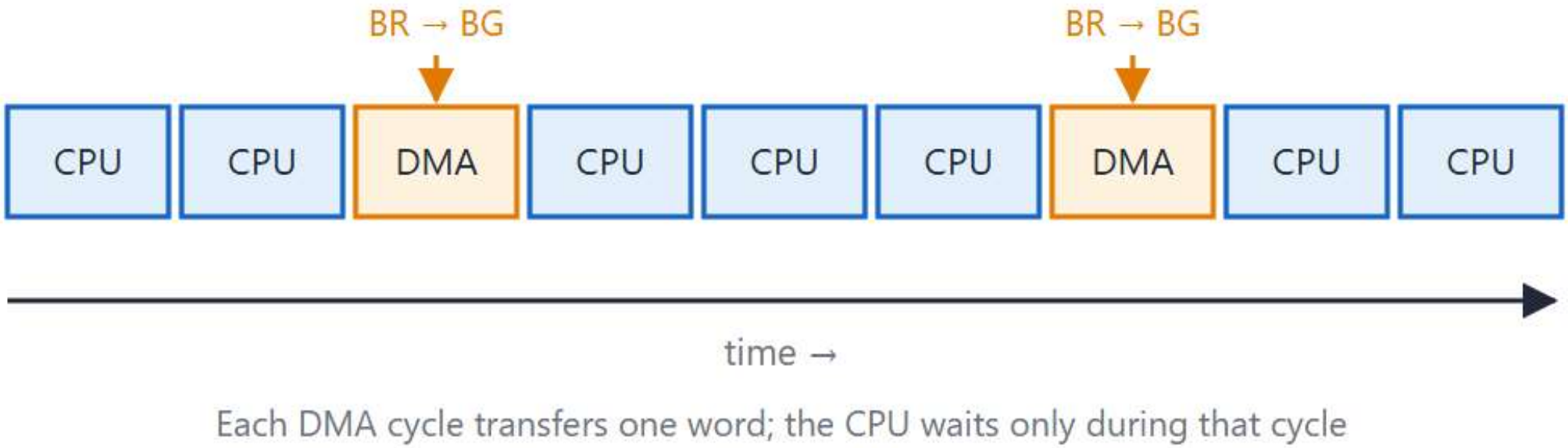


CPU bus signals for DMA transfer. Adapted from M. Morris Mano, Computer System Architecture, Fig. 11-17.

Burst transfer and cycle stealing

- **Burst transfer:** the controller keeps the bus for a whole block in one go; fast, used for fast devices such as disks
- **Cycle stealing:** the controller takes the bus for one word at a time and gives it back
- Cycle stealing is the most common method: one stolen bus cycle per word
- **Effect on the CPU:** slightly slowed (held off the bus for one cycle per steal) but it keeps executing between steals
- Far more efficient than interrupt-initiated I/O for large blocks

DMA cycle stealing

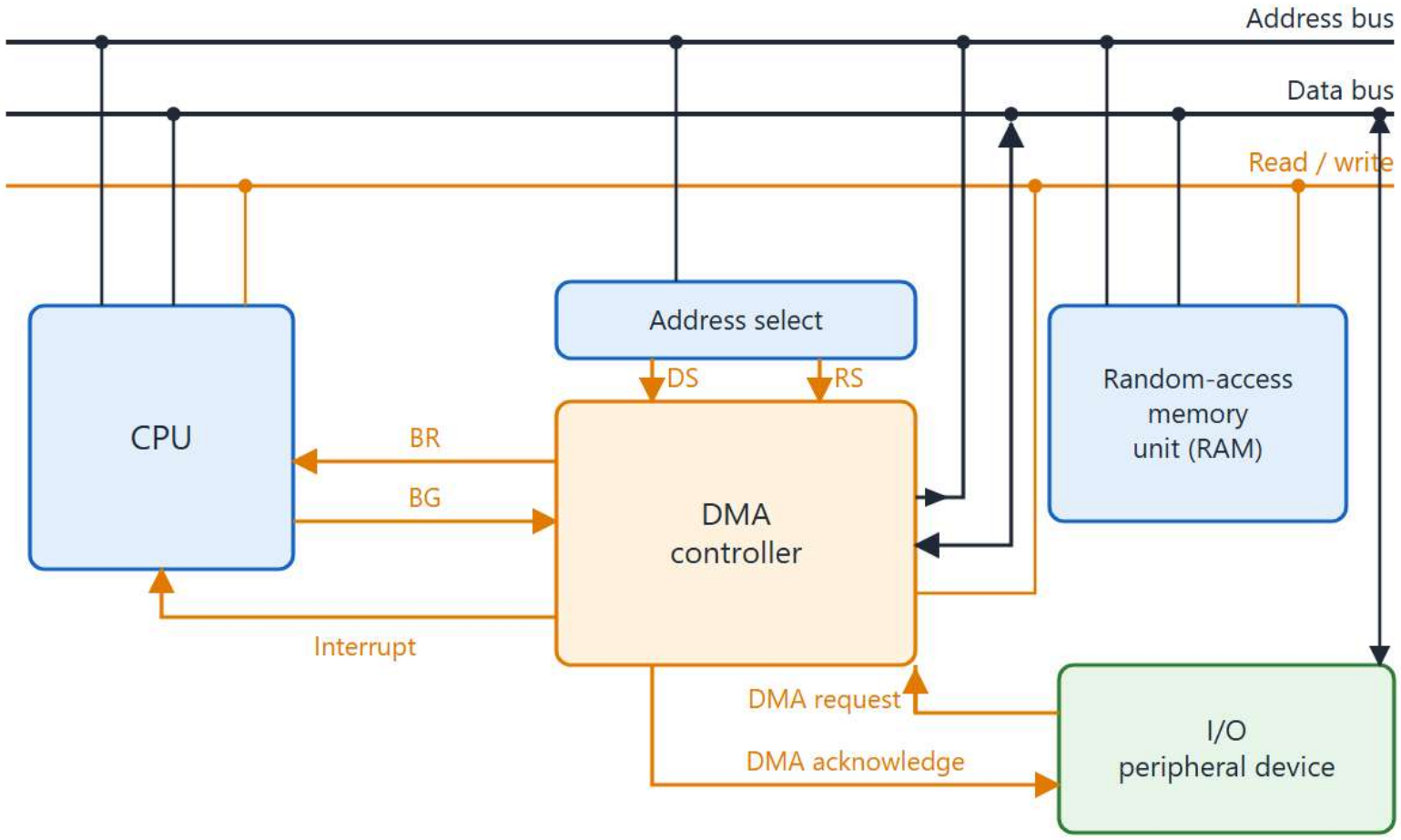


DMA cycle stealing — one bus cycle is stolen per word transferred

Sequence for each stolen cycle

- **1.** The DMA controller asserts BR
- **2.** The CPU finishes its current bus cycle and asserts BG
- **3.** The controller transfers one word (address from AR, data to or from the device)
- **4.** The controller increments AR, decrements WC and releases the bus
- **5.** The CPU resumes; if WC ≠ 0, repeat from step 1 after the next device-ready signal

DMA transfer in a computer system



DMA transfer in a computer system. Adapted from M. Morris Mano, Computer System Architecture, Fig. 11-19.

DMA compared with the other modes

Property	Programmed I/O	Interrupt-initiated	DMA
CPU during transfer	Busy-wait (polling)	Runs other code; ISR per byte	Runs other code; 1 interrupt total
Transfer unit	1 byte per poll	1 byte per interrupt	Block (many bytes)
CPU involvement	Every byte	Every byte (ISR)	Initialise + completion only
Hardware needed	Interface only	Interface + interrupt logic	Interface + DMA controller
Best for	Slow/simple devices	Medium-speed devices	High-speed block devices (disk, network)

Comparison of the three I/O transfer modes

5.5 Memory Hierarchy

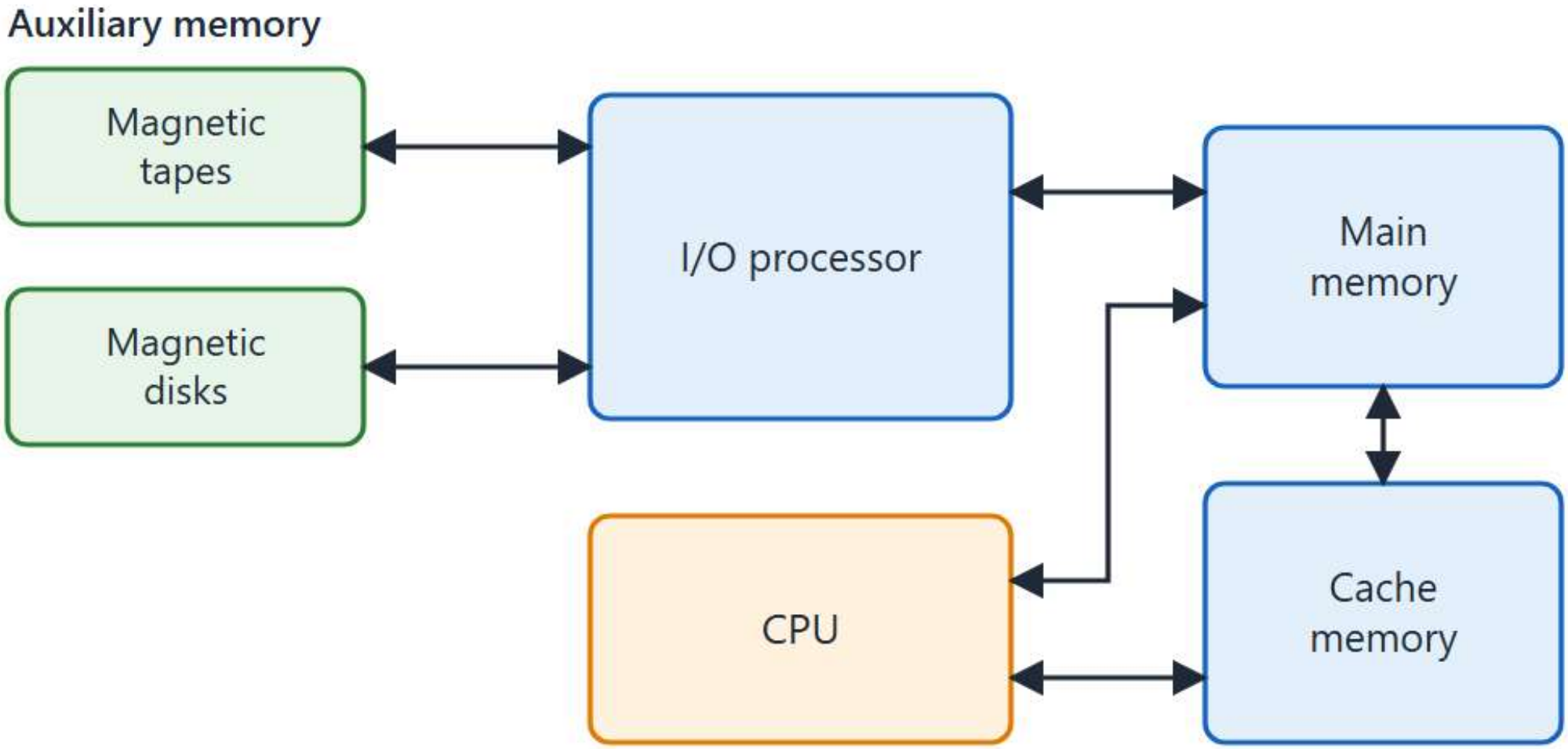
IN SIMPLE TERMS

Fast memory is expensive and small; cheap memory is slow and large. Computers combine several kinds (registers, cache, main memory, disk, tape) and keep the data needed right now in the fastest levels.

Why a hierarchy

- No single technology gives unlimited capacity, zero access time and zero cost
- A **memory hierarchy** uses several levels, each faster and costlier than the one below
- Going up: speed and cost per bit rise; practical capacity falls
- Goal: the impression of fast, large, cheap memory
- Keep the most-used data in the fast levels and the rest in slower ones

Memory hierarchy



Memory hierarchy in a computer system. Adapted from M. Morris Mano, Computer System Architecture, Fig. 12-1.

The five levels

- **Magnetic tape:** removable, sequential, very large, very slow; archival and backup
- **Magnetic disk:** random access, large, slower than main memory; primary auxiliary storage
- **Main memory (RAM/ROM):** directly accessed by the CPU; ~100–700 ns
- **Cache memory:** very small and fast (~10–100 ns); holds the active parts of programs and data
- **CPU registers:** fastest; a few words inside the processor; one clock cycle

Typical characteristics at each level

Level	Typical access time	Relative cost/bit	Capacity
CPU registers	1 clock cycle (<1 ns)	Highest	Bytes
Cache memory	~10–100 ns	High	KB – MB
Main memory	~100–700 ns	Medium	MB – GB
Magnetic disk	~5–20 ms	Low	GB – TB
Magnetic tape	Seconds (sequential)	Lowest	TB+

Typical characteristics at each hierarchy level

Cache and the access-time ratios

- Cache is about **7× faster** than main memory (100 ns versus 700 ns)
- Auxiliary memory (disk) is about **1000× slower** than main memory
- Block sizes: auxiliary memory 256–2048 words; cache 1–16 words
- Cache sits between CPU and main memory; the CPU looks there first
- **Hit:** data found, obtained quickly. **Miss:** fetched from main memory, often a whole block
- This works because of **locality of reference**: programs reuse recently used locations

Auxiliary memory, IOP and multiprogramming

- The CPU cannot access auxiliary memory directly; the **I/O processor (IOP)** moves data using DMA
- The CPU asks the IOP for data, then continues with another task
- The IOP interrupts the CPU when the transfer is done
- **Multiprogramming:** several programs at different levels at once: one running (using cache), others in main memory, others on disk
- The memory management system (software + hardware) moves programs between levels
- This keeps the CPU, IOP and I/O devices all busy

5.6 Main Memory

IN SIMPLE TERMS

Main memory is built from RAM chips (read and write, lost at power-off) and ROM chips (read-only, kept at power-off). The address decides which chip answers: low address bits pick a byte inside a chip, high bits pick the chip.

RAM: random access memory

- Most of main memory is RAM, in two technologies
- **Static RAM (SRAM):** each bit in a flip-flop; stable while powered; faster, more silicon per bit
- **Dynamic RAM (DRAM):** each bit as charge on a MOS capacitor; must be **refreshed** every few ms; higher density, lower power per bit
- RAM is **volatile**: contents are lost at power-off
- Used for programs and data that change while the computer runs

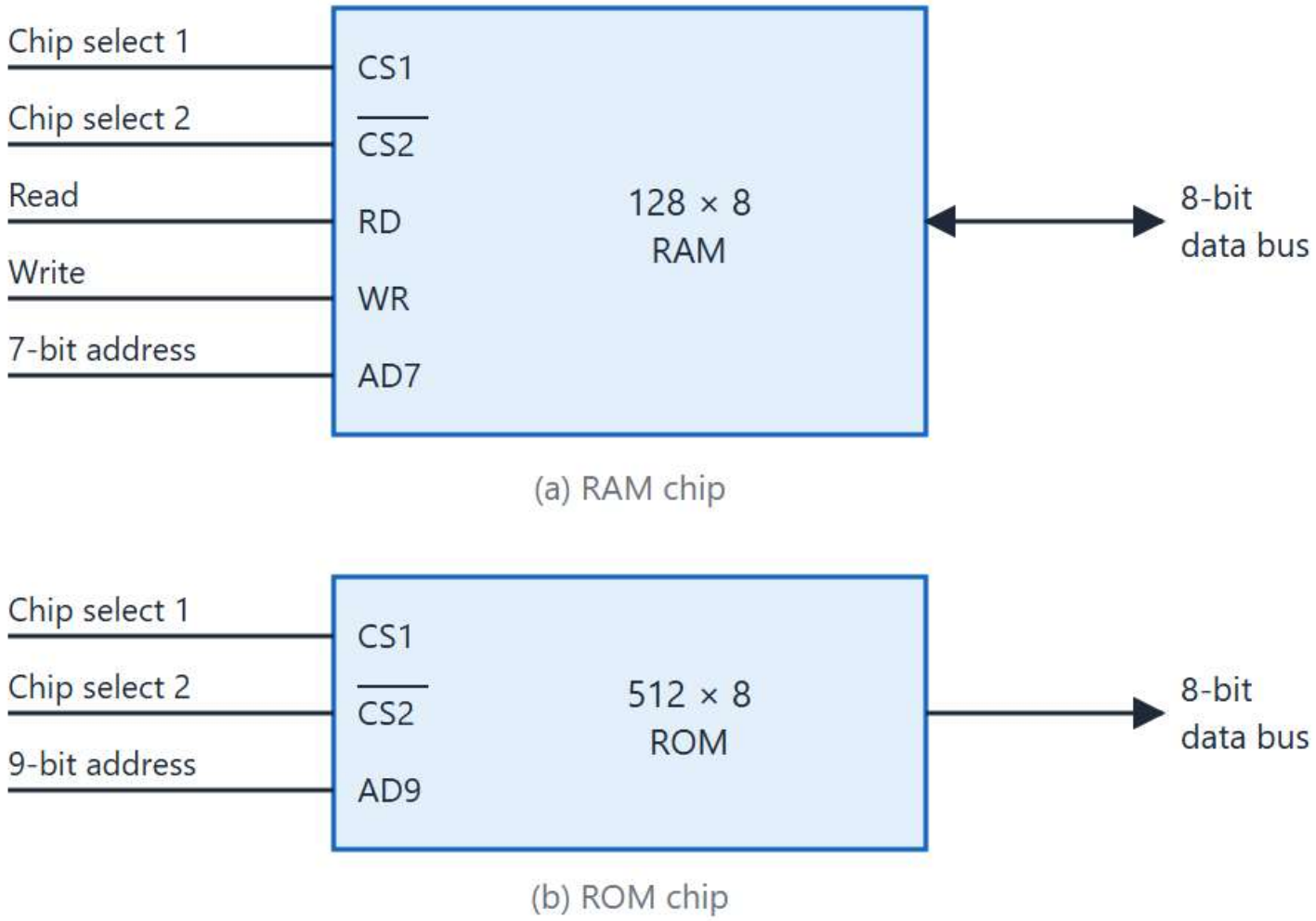
ROM: read-only memory

- ROM is **non-volatile**: it keeps its contents when power is off
- It holds programs that must always be present: the **bootstrap loader**, firmware, constant tables
- At power-on the CPU sets its PC to the bootstrap loader address
- The bootstrap loads the OS from disk into RAM, then hands control to the OS

RAM and ROM chip structure

- A RAM chip has a cell array, a bidirectional data bus, and address, read and write inputs
- Chip selects: CS_1 and CS_2'; the chip is active only when $CS_1 = 1$ and $CS_2' = 0$
- Three-state bus buffers let several chips share one data bus
- A ROM chip is organised similarly but has no WR input: its data bus is output-only
- For the same chip area a ROM can hold more bits than a RAM, because its cells are simpler

RAM and ROM chips



Typical RAM and ROM chips. Adapted from M. Morris Mano, Computer System Architecture, Figs. 12-2 and 12-3.

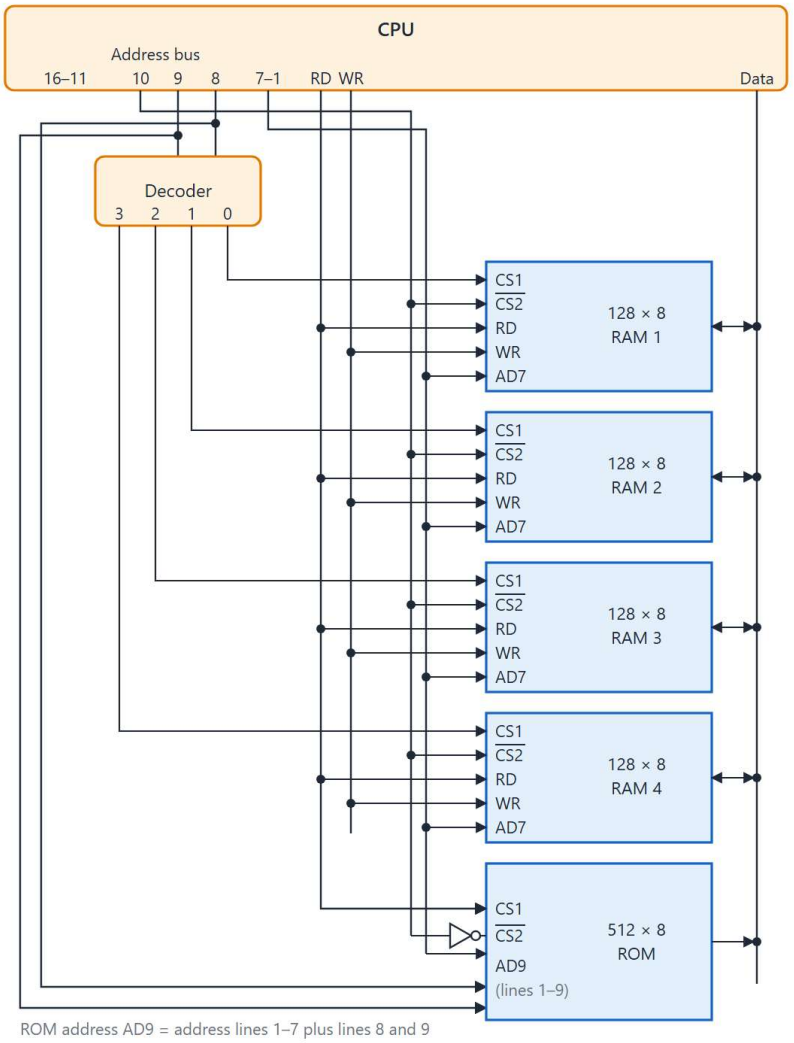
Memory address map

Example: a 1 KB memory from four 128×8 RAM chips and one 512×8 ROM chip.

Component	Hex address range	A_9–A_8	A_{10}
RAM 1	0000–007F	00	0
RAM 2	0080–00FF	01	0
RAM 3	0100–017F	10	0
RAM 4	0180–01FF	11	0
ROM	0200–03FF	xx	1

Memory address map for microcomputer example. Adapted from M. Morris Mano, Computer System Architecture, Table 12-1.

Memory connection to the CPU



- Low-order lines A_1–A_7 select one of 128 bytes inside each RAM chip
- A 2 × 4 decoder on A_8 and A_9 picks one RAM chip when $A_{10} = 0$
- $A_{10} = 1$ enables the ROM
- Design rule: exactly one chip selected, all others in high impedance

Memory connection to the CPU. Adapted from M. Morris Mano, Computer System Architecture, Fig. 12-4.

5.7 Auxiliary Memory

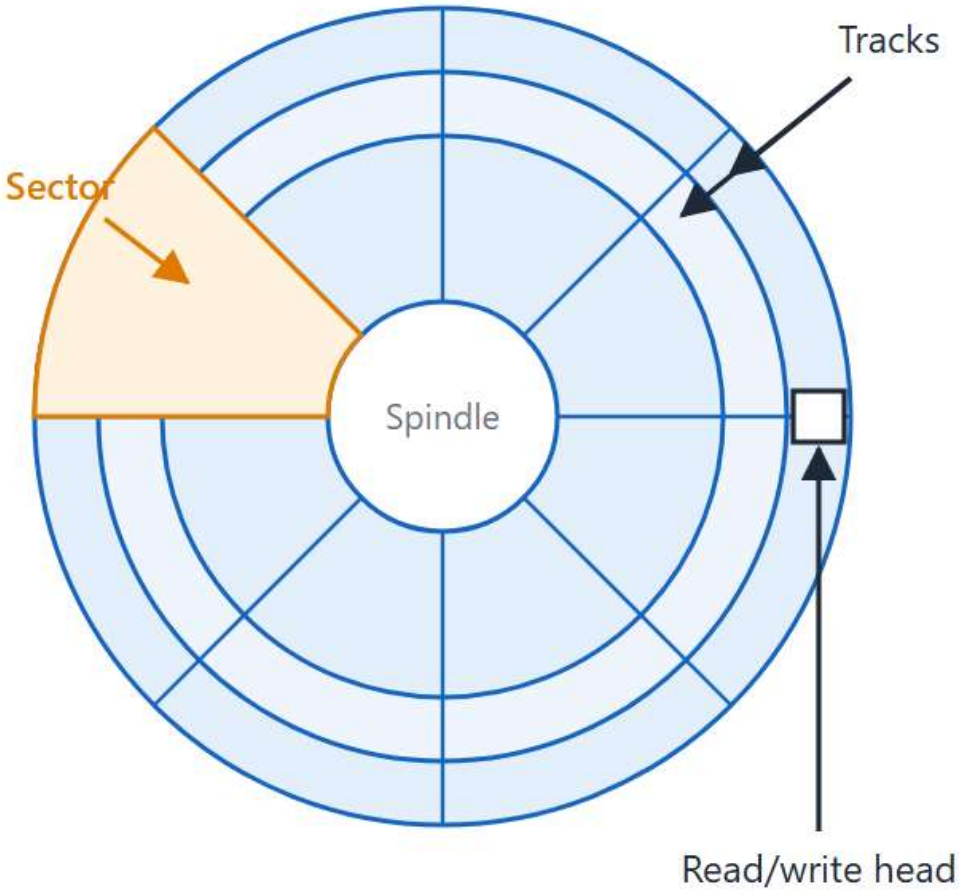
IN SIMPLE TERMS

Auxiliary memory (disk, tape) stores everything not needed right now. It is cheap and huge but slow, because a head must first move to the right place, so data is always moved in large blocks.

General characteristics

- Holds programs and data not needed right now: large, low cost, slow
- Five parameters: access mode, access time, transfer rate, capacity, cost
- **Seek time:** positioning the head over the right track; the dominant part, usually several ms
- **Transfer time:** reading or writing once the head is in place
- Seek time ≫ transfer time, so data is organised in **records**
- Reads and writes use complete records, to spread the seek overhead

Magnetic disk



- Data is stored along concentric **tracks**
- Each track is divided into **sectors**, the smallest unit transferred
- A disk is addressed by disk number, surface, sector number and track
- Seek moves the head; rotational latency waits for the sector

Magnetic disk: concentric tracks divided into sectors. Adapted from M. Morris Mano, Computer System Architecture, Fig. 12-5.

Hard disks and floppy disks

- **Hard disks:** permanently attached, metal surfaces, high speed and capacity
- One movable head per surface is cheaper; one head per track is faster but costlier
- **Floppy disks:** removable, flexible plastic coated with magnetic material
- Two standard sizes: 5.25-inch and 3.5-inch
- Used for software distribution and small data files

Magnetic tape

- A strip of plastic coated with magnetic material
- Bits are recorded across 7 or 9 parallel tracks, one character per column; one head per track
- Access is **sequential**: data only in the order stored
- A drive cannot stop and start between characters, so data is grouped in **records** separated by blank **inter-record gaps**
- The tape stops in a gap and speeds up before the next record
- Each record has an identification pattern at its start and end

Magnetic disk compared with tape

Key difference: a disk gives random access to any sector; a tape gives sequential access only.

Property	Magnetic disk	Magnetic tape
Access mode	Random (sector addressable)	Sequential only
Access time	Seek + rotational latency (ms)	Must wind to correct record (seconds)
Transfer unit	Sector	Record
Typical use	Active programs and data, OS	Backup, archival, removable storage
Removable?	Floppy yes; hard disk no	Yes

Comparison of magnetic disk and tape

5.8 Associative Memory

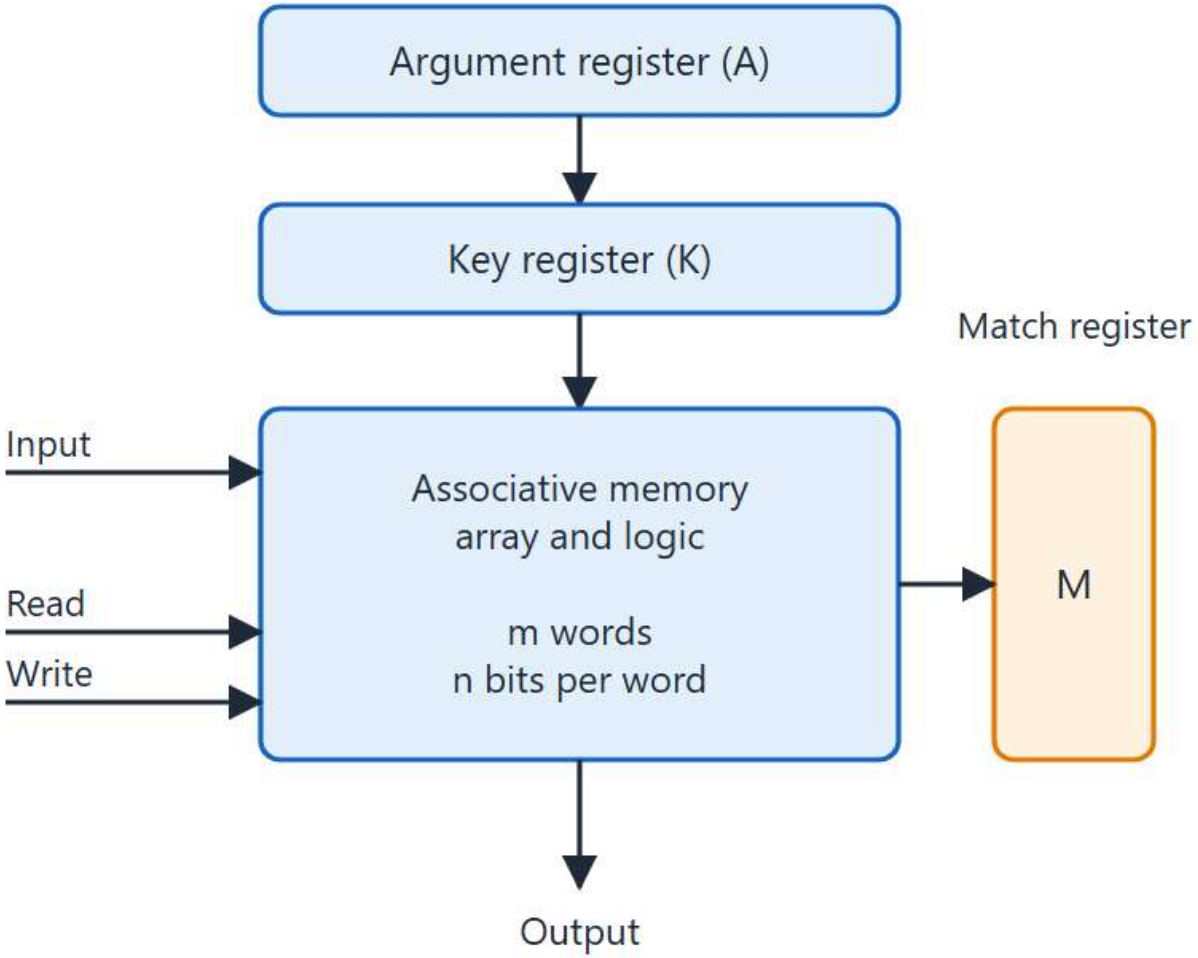
IN SIMPLE TERMS

A normal memory is asked “what is at address 25?”. An associative memory is asked “where is the word that looks like this?” and checks every word at the same time.

Basic concept

- Many tasks search a table: a symbol table, an account number, a cache block
- Ordinary RAM searches sequentially, one word at a time
- **Associative memory** (content-addressable memory, CAM) searches all words at once in hardware
- You supply a data pattern (the **argument**), not an address; every stored word is compared in parallel
- Matching words are flagged in the match register; read by addressing only those words
- Costlier per bit than RAM; used where search time is critical: cache tag arrays, TLBs

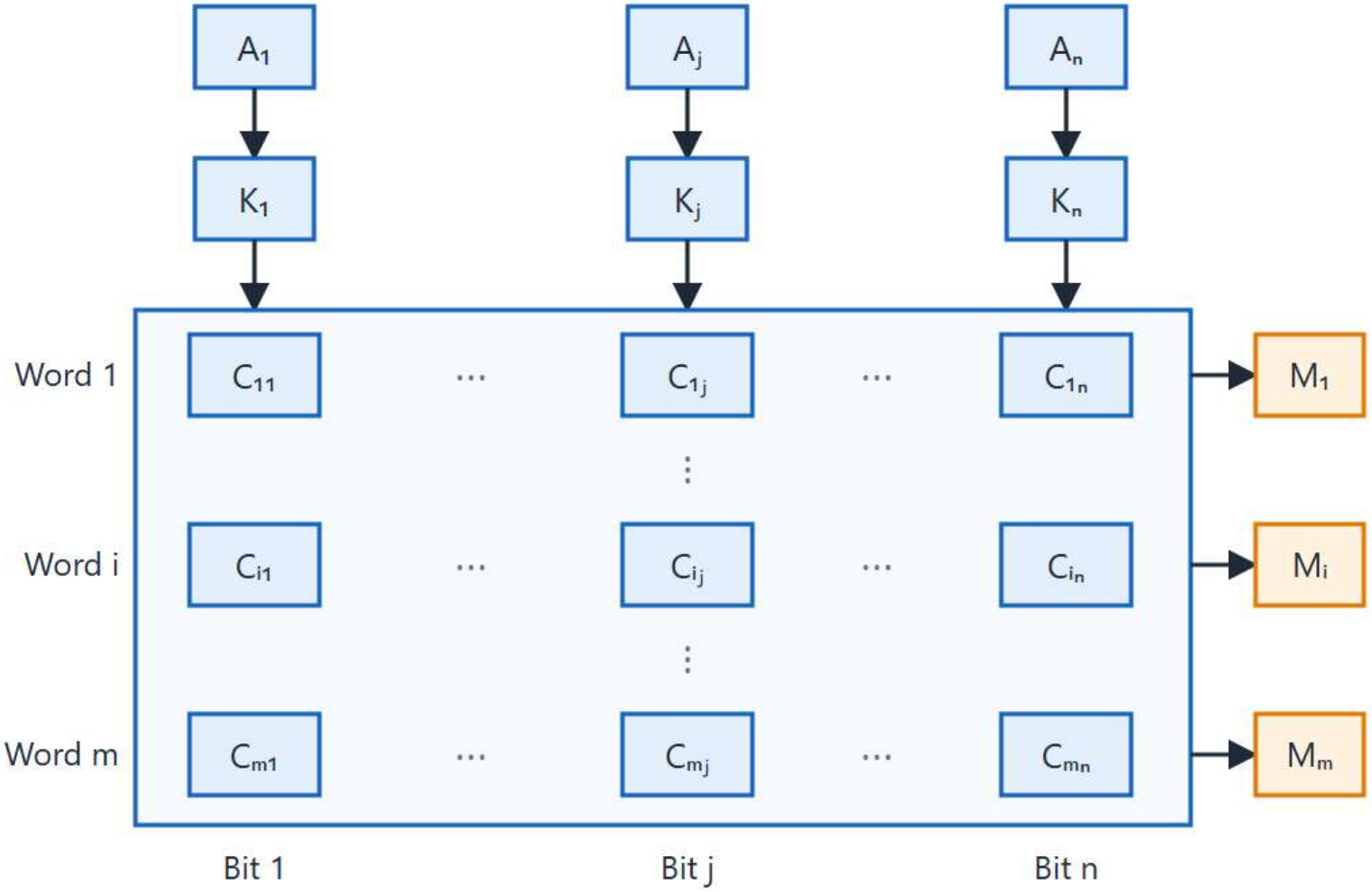
Associative memory block diagram



- **Argument register A** (n bits) holds the search pattern
- **Key register K** (n bits) is a mask: $K_j = 1$ compares bit j, $K_j = 0$ ignores it
- **Memory array:** m words × n bits of CAM cells
- **Match register M** (m bits): $M_i = 1$ if word i matched all unmasked bits

Block diagram of associative memory. Adapted from M. Morris Mano, Computer System Architecture, Fig. 12-6.

Associative memory cell array



Associative memory of m words, n cells per word. Adapted from M. Morris Mano, Computer System Architecture, Fig. 12-7.

Cell logic and match equations

$$x_j = A_j F_{ij} + A_j' F_{ij}'$$
$$term_j = x_j + K_j'$$
$$M_i = (x_1 + K_1')(x_2 + K_2') \cdots (x_n + K_n')$$

- Cell C_{ij} has a flip-flop F_{ij} for storage, plus read, write and match circuitry
- $x_j = 1$ if A_j equals F_{ij}; otherwise 0
- If $K_j = 0$ then $K_j' = 1$, so $term_j = 1$ and bit j is ignored
- Word i matches if and only if all n terms equal 1

Worked example: three-bit match

A = 101 111100 and K = 111 000000: only the three leftmost bits are compared ($K_j = 1$ for j = 1, 2, 3).

Word	Stored bits	Three-bit match?	M_i
Word 1	100 111100	100 ≠ 101: no match	0
Word 2	101 000001	101 = 101: match	1

Read and write operations

- **Read:** after a search, M shows which words matched
- If exactly one word matched, its data goes to the output automatically
- If several matched, they are read one at a time by applying a read signal to each matched word line
- **Write:** address each location in turn (acting as a RAM), or scan the tag register for an unused location (tag bit = 0) and write there

5.9 Cache Memory

IN SIMPLE TERMS

A cache keeps copies of recently used words in a small, fast memory next to the CPU. Because programs reuse the same instructions and data again and again, most requests are found in the cache.

Locality of reference

- Over a short interval, memory accesses stay in a small, localised region
- **Instruction loops:** the same instructions are fetched repeatedly
- **Subroutine calls:** a subroutine's instructions sit in one address range
- **Array and table accesses:** data in contiguous or nearby locations
- **Hit:** the word is already in the cache
- **Miss:** fetch from main memory and load a block of adjacent words

Hit ratio and average access time

$$h = hits / (hits + misses)$$

$$T_{avg} = h \cdot T_c + (1 - h) \cdot (T_c + T_m)$$

- T_c is the cache access time; T_m is the main memory access time
- On a miss the cache is checked first, so a miss costs $T_c + T_m$
- Hit ratios of 0.9 and higher are common

Worked example: average access time

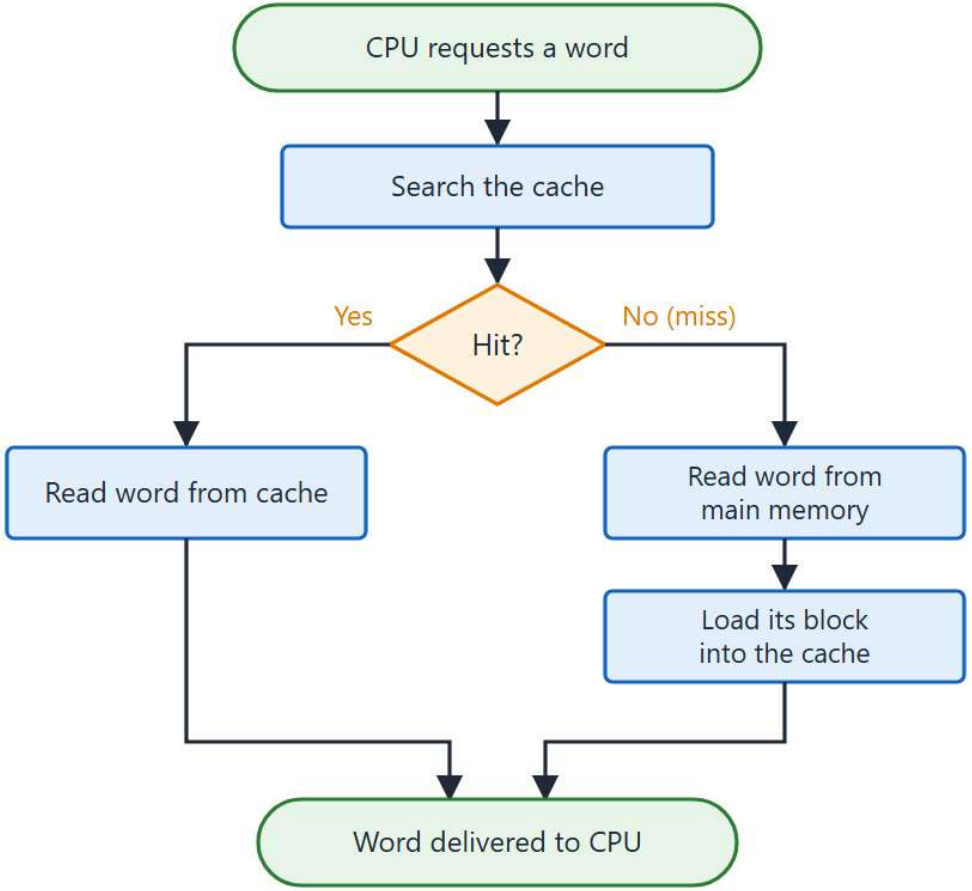
$$T_c = 100 \text{ ns}, \quad T_m = 1000 \text{ ns}, \quad h = 0.9$$

$$\text{Miss cost} = T_c + T_m = 1100 \text{ ns}$$

$$T_{avg} = 0.9 \times 100 + 0.1 \times 1100 = 200 \text{ ns}$$

- Main memory alone takes 1000 ns
- With the cache the average is 200 ns: **five times faster**

Cache hit and miss



Average access time (miss checks cache first):

$$T = h \cdot Tc + (1 - h)(Tc + Tm)$$

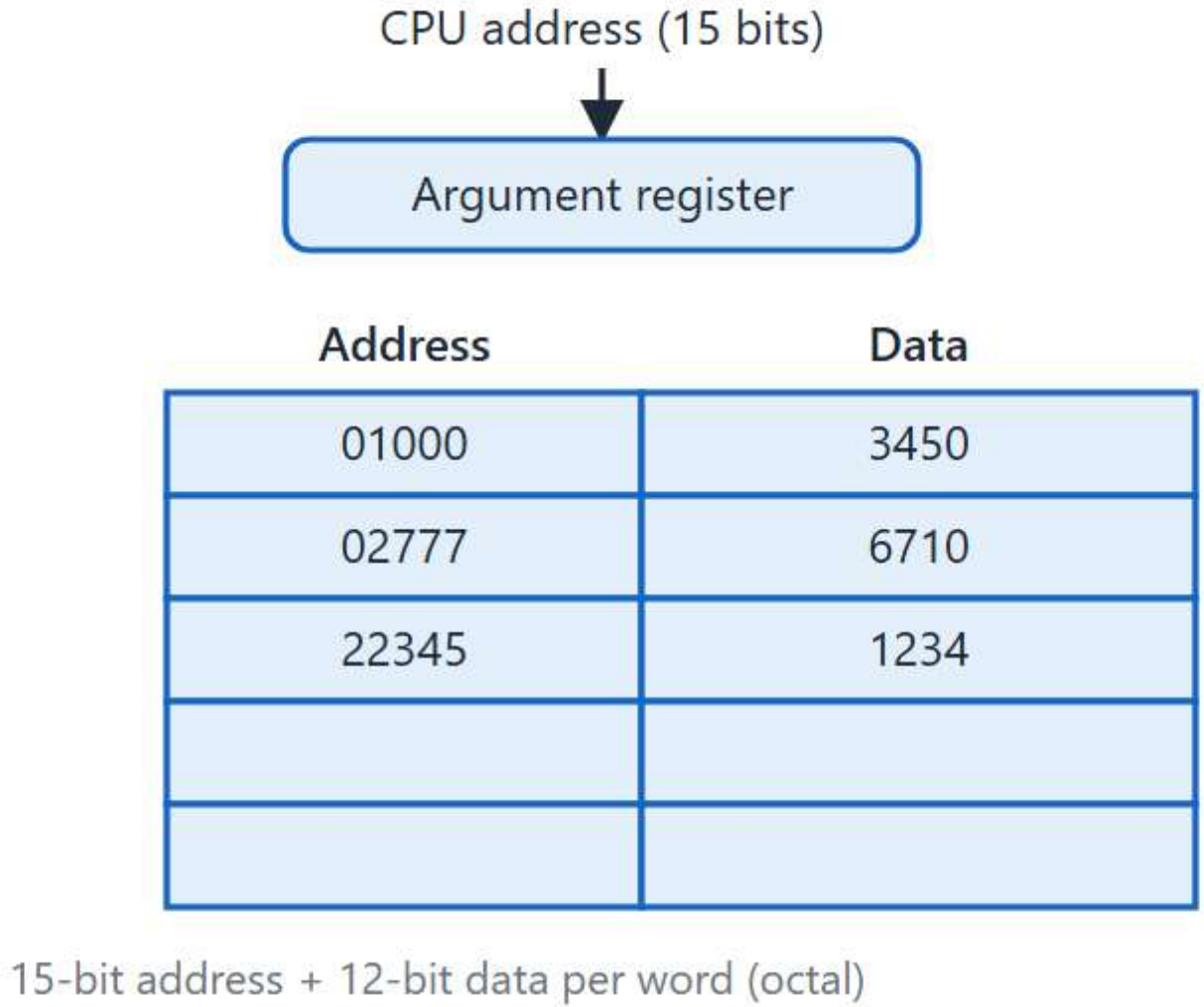
Cache hit / miss operation and hit ratio formula

- **Hit:** the requested word is in the cache
- **Miss:** fetch it from main memory, loading a block of adjacent words
- Hit ratio h = hits / (hits + misses)
- Hit ratios of 0.9 and higher are common

Mapping method 1: associative

- The mapping decides where a main memory block lives in the cache and where to look for it
- Example used: 32K × 12 main memory, 512 × 12 cache, 15-bit CPU address
- **Advantage:** any block can go to any cache location: no conflict misses
- **Disadvantage:** the associative search hardware is expensive; suited only to small caches

Associative mapping cache



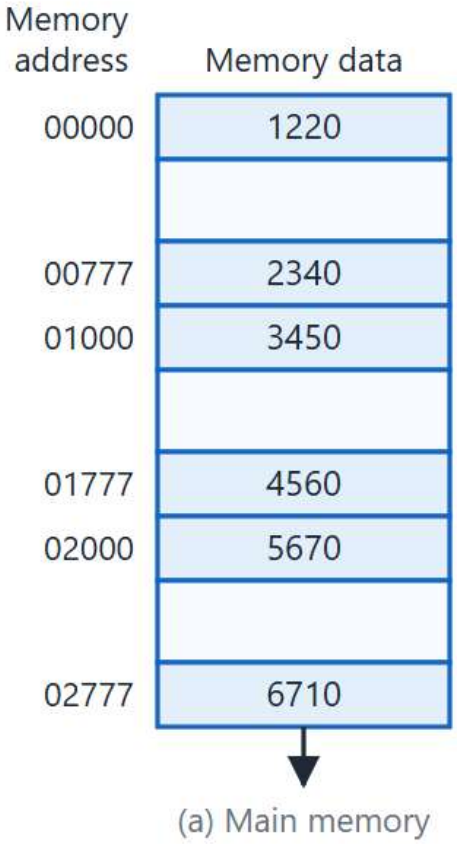
- The cache stores the full 15-bit address and the 12-bit data word together
- The CPU address is searched against all entries at once
- A match is a hit and the data is returned
- All numbers are in octal

Associative mapping cache (all numbers in octal). Adapted from M. Morris Mano, Computer System Architecture, Fig. 12-11.

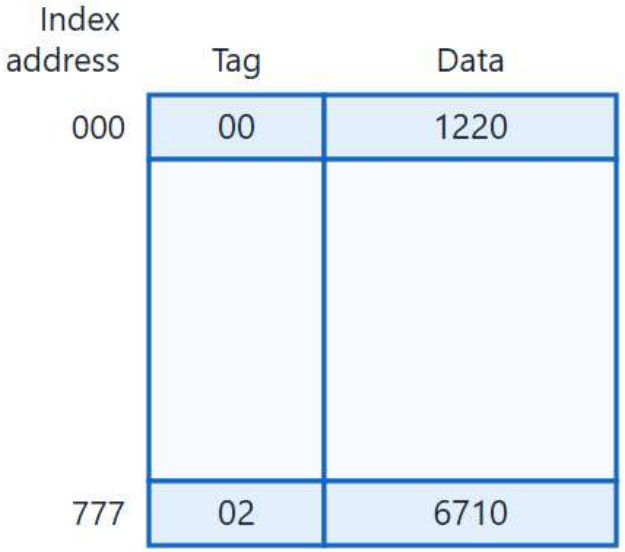
Mapping method 2: direct

- The 15-bit address splits into a **tag** field (6 bits) and an **index** field (9 bits)
- The index selects one of 512 cache lines
- The tag stored there is compared with the address tag to confirm a hit
- **Advantage:** fast; no search, only one tag comparison
- **Disadvantage:** conflict misses: two addresses with the same index and different tags keep evicting each other
- With 8-word blocks, the 9-bit index = 6-bit block field + 3-bit word field; a miss loads a whole block of 8 words

Direct mapping cache



(a) Main memory



(b) Cache memory

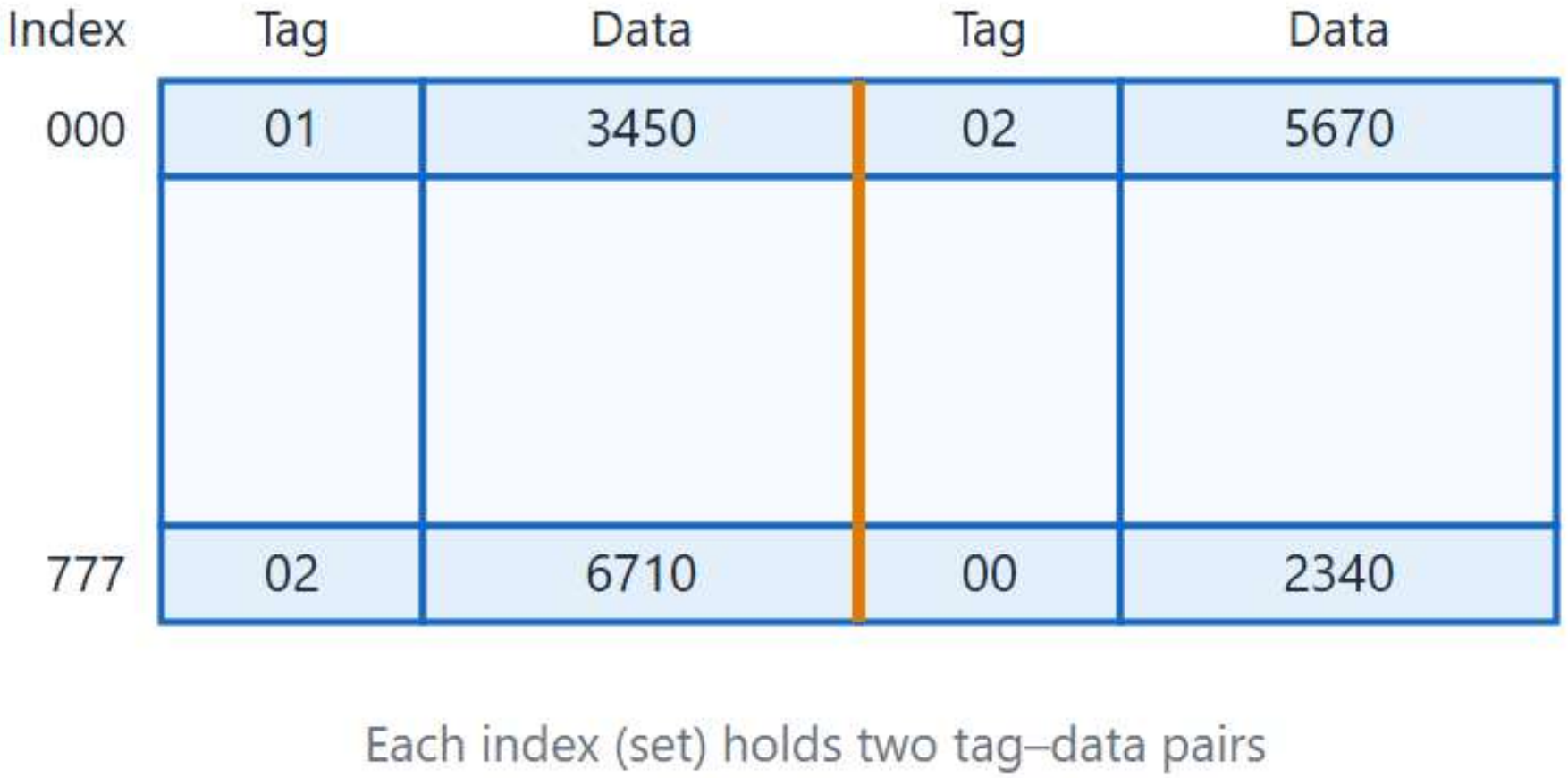
CPU address = tag (6 bits) + index (9 bits)

Direct mapping cache organization. Adapted from M. Morris Mano, Computer System Architecture, Fig. 12-13.

Mapping method 3: set-associative

- A compromise: the cache is organised into sets of k locations
- The index selects a **set**; the k tag–data pairs in it are searched associatively
- Two-way example: 512 sets, each with two tag–data pairs
- Cache word = 2 × (6 + 12) = 36 bits: a 512 × 36 cache holding 1024 words of main memory
- **Advantage:** k words with the same index can coexist: fewer conflict misses
- **Disadvantage:** a little more complex than direct mapping; cost between direct and fully associative

Two-way set-associative cache



Two-way set-associative mapping cache. Adapted from M. Morris Mano, Computer System Architecture, Fig. 12-15.

Comparison of the three mapping methods

Property	Associative	Direct	Set-associative (k = 2)
Address partition	Full 15-bit address stored	Tag (6b) + Index (9b)	Tag (6b) + Index (9b)
Lookup method	Parallel associative search	Index → compare 1 tag	Index → compare k tags
Conflict misses	None	High (1 block per index)	Low (k blocks per index)
Hardware cost	Highest	Lowest	Moderate

Comparison of the three mapping methods (32K × 12 memory, 512 × 12 cache, 15-bit address)

Replacement algorithms

- When a miss finds the target line or set full, a block must be evicted
- **Random:** choose any block; simple hardware, acceptable performance
- **FIFO (first-in, first-out):** evict the block that has been in the set longest; a few extra bits per line
- **LRU (least recently used):** evict the block used least recently; best hit ratio in practice; usage order tracked per set

Write policies and the valid bit

- **Write-through:** write to cache and main memory together; memory always up to date (DMA gets current data); more bus traffic
- **Write-back (copy-back):** write to cache only; mark the block with a **dirty bit**; update memory when the block is evicted
- Write-back means fewer bus writes, but main memory can be stale
- Each cache word has a **valid bit**, cleared at power-up: every first access is a miss
- The valid bit is set to 1 when a block is loaded, so old data is never supplied

Unit 5 summary (1 of 2)

- **5.1 I/O interface:** resolves four CPU–peripheral differences; four command types; isolated or memory-mapped addressing
- **5.2 Modes of transfer:** programmed I/O polls a flag; interrupt-initiated I/O lets the device interrupt; DMA moves whole blocks
- **5.3 Priority interrupt:** highest priority first, by software polling, daisy chain or parallel hardware (interrupt register, mask register, priority encoder)
- **5.4 DMA:** the controller (AR, WC, control/status) moves a block by BR/BG cycle stealing and interrupts once at WC = 0
- **5.5 Memory hierarchy:** registers, cache, main memory, auxiliary memory; fast and costly at the top, large and cheap at the bottom

Unit 5 summary (2 of 2)

- **5.6 Main memory:** SRAM, DRAM (needs refresh) and ROM (non-volatile); the address map makes each address select exactly one chip
- **5.7 Auxiliary memory:** disk (random access, tracks and sectors) and tape (sequential); seek time dominates
- **5.8 Associative memory:** searches all words in parallel using A, K and M; match logic in every cell
- **5.9 Cache memory:** works by locality of reference; associative, direct or set-associative mapping; random, FIFO or LRU replacement; write-through or write-back
- **Key applications:** DMA for disk and network I/O; ROM bootstrap loader starts the OS; associative memory in cache tag arrays and TLBs